

Universidad Alfonso X El Sabio

Asignatura: Gestión de Datos · Curso 2025/26

Informe técnico de proyecto final

SalesHealth: construcción de un Data Warehouse y segmentación de clientes mediante CLTV y PCA+KMeans

Autor: Alejandro Barreche
Titulación: Grado en Ingeniería Matemática
Entrega: Mayo 2026

Resumen

Este informe documenta **SalesHealth**, entrega final de la asignatura Gestión de Datos (UAX, curso 2025/26). Sobre un dataset relacional sintético del sector salud y bienestar (5 750 clientes, 20 000 tickets, 42 555 líneas, 2 330 devoluciones, ventana 2020-01–2026-01) se construye un *Data Warehouse* en estrella sobre PostgreSQL, se calcula el CLTV por cliente y se segmenta la base mediante PCA y KMeans. El ETL queda automatizado por seis notebooks numerados, con cobertura de `pytest` sobre la integridad referencial del DWH y la consistencia de la fórmula de CLTV.

1 Introducción

El proyecto parte de un dataset sintético del sector salud y bienestar distribuido como punto de partida para la entrega final. Reproduce un escenario retail clásico (50 productos, una red de tiendas, catálogo de promociones, devoluciones con motivo) sobre una ventana de 72 meses (2020–2025, ambos inclusive). Los datos crudos están cargados en el esquema `public` de PostgreSQL 16 en Docker (`localhost:5433`, BD `saleshealth`). La tabla 1 reúne las cifras verificadas contra el DWH y los *parquet* de analítica.

Tabla 1: Cifras clave verificadas (DWH + data/processed/).

Productos	50	Líneas de venta	42 555
Clientes	5 750	Líneas devueltas	2 330
Tickets de venta	20 000	Ingresos brutos	9,68 M€
Ingresos netos	9,40 M€	Ventana	72 meses

Objetivos. (i) Diseñar un modelo Entidad–Relación sobre `public` y un modelo dimensional en estrella sobre `dwh`. (ii) Implementar un ETL reproducible `public`→`staging`→`dwh`. (iii) Calcular el CLTV por cliente, $CLTV = Ingresos_t \times Margen_t \times Frecuencia_t \times R_t$, junto con *Recencia*, *AOV* y *tasa de devolución*. (iv) Segmentar la base mediante *PCA* + *KMeans*. (v) Documentar todo en un informe y un dashboard.

Stack. Python 3.11 en entorno `conda` UAX; PostgreSQL 16; `pandas`, `SQLAlchemy` 2.0, `scikit-learn`; notebooks Jupyter como orquestador; Flask para el dashboard; `pytest`.

2 Modelo de datos

El esquema `public` agrupa 17 tablas crudas en cinco bloques: *producto* (`product`, `central_product`, `category`, `brand`, `offer`, `product_offer`); *almacén* (`warehouse`, `warehouse_location`, `central_inventory`, `inventory`); *tienda y geografía* (`store`, `city_zone`); *cliente y ventas* (`customer`, `sale`, `sale_item`); y *devoluciones* (`return_item`, `return_reason`). Las relaciones clave son `sale`1–`N``sale_item`, `sale`–`N`1 `customer` y `store`, `sale_item`–`N`1 `product`, y `sale_item`1–`N``return_item` (figura 1).

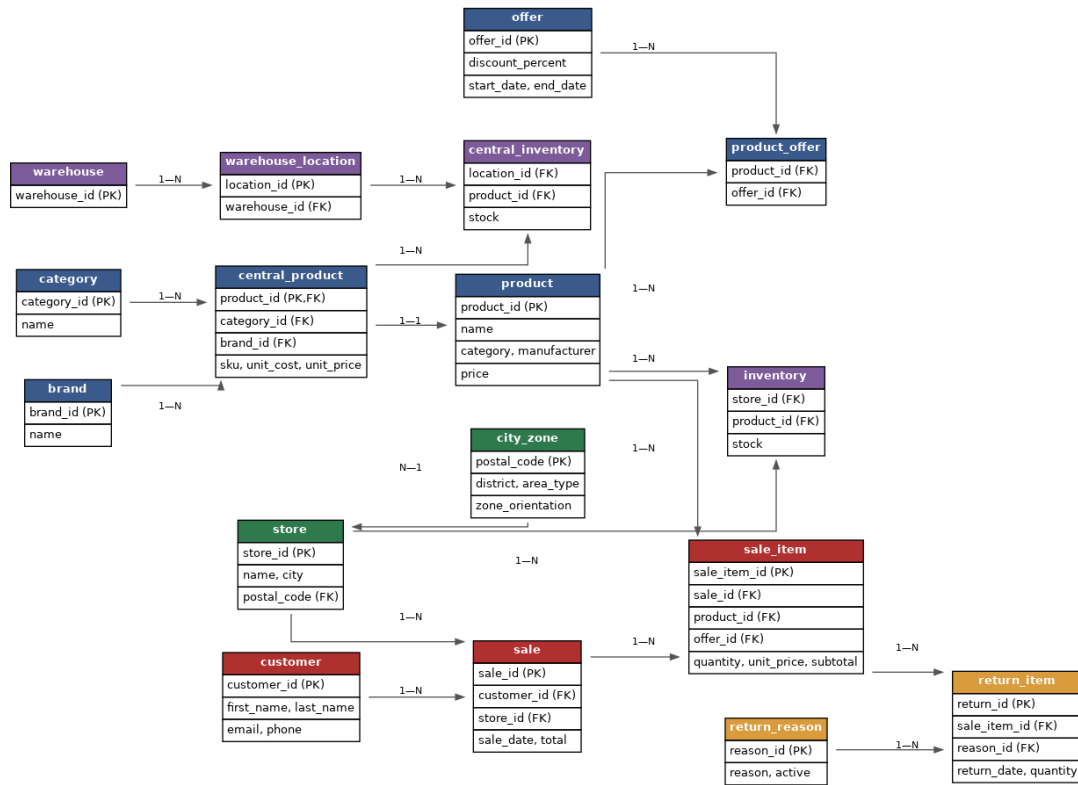


Figura 1: Esquema public: 17 tablas agrupadas en cinco bloques funcionales.

El esquema dwh sigue un patrón clásico en estrella con seis dimensiones (`dim_date`, `dim_customer`, `dim_product`, `dim_store`, `dim_offer`, `dim_return_reason`) y dos hechos: `fact_sales` (grano = línea de venta, 42 555 filas, métricas `quantity`, `unit_price`, `unit_cost`, `subtotal`, `cost`, `margin`) y `fact_returns` (grano = línea devuelta, 2 330 filas, métricas `quantity` y `returned_value`). Véase la figura 2.

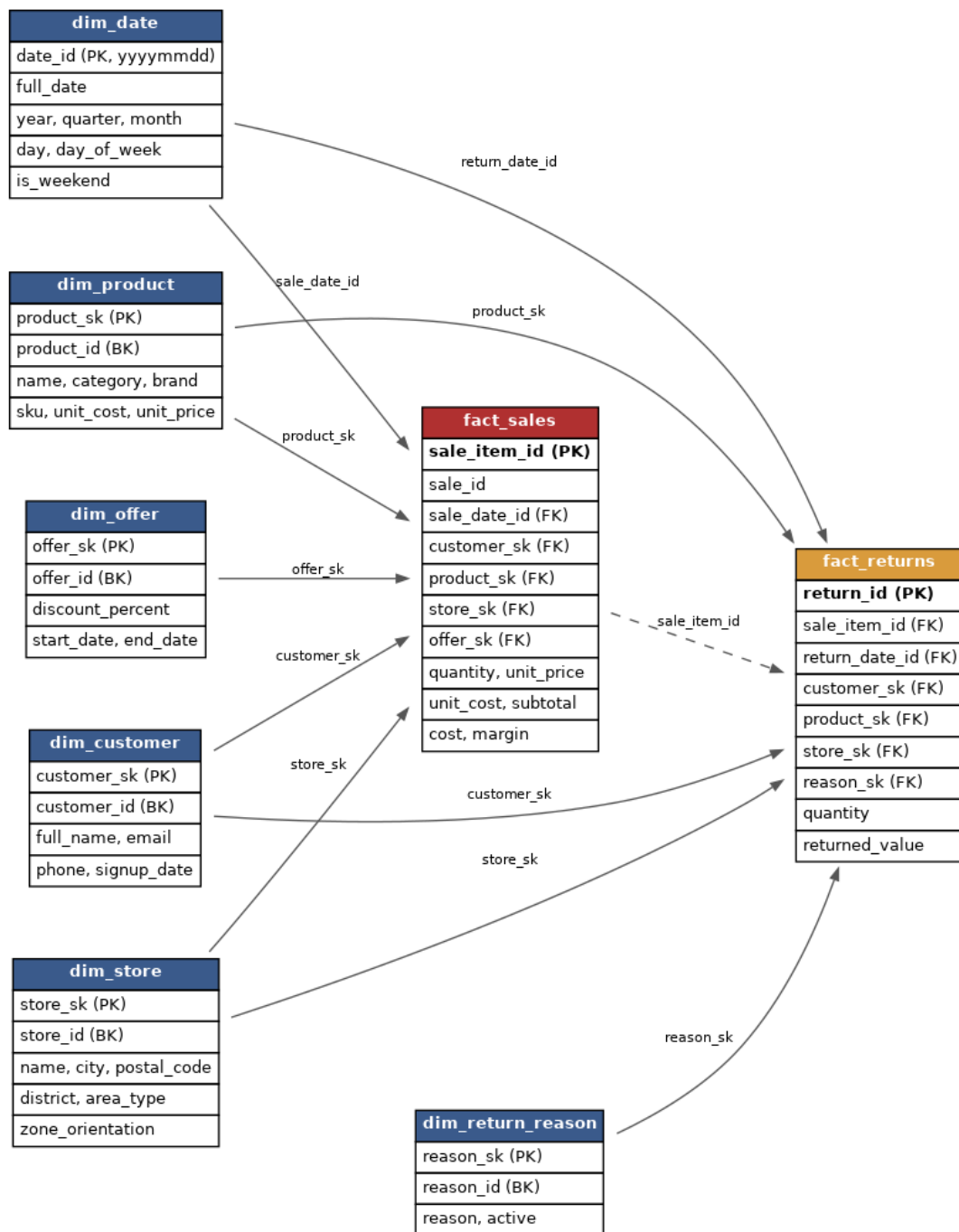


Figura 2: Esquema dwh: seis dimensiones más dos hechos. La línea discontinua representa el *drill-through* de fact_returns hacia fact_sales por sale_item_id.

Decisiones de diseño

Todas las dimensiones tienen una *surrogate key* *_sk (serial) y conservan la clave de negocio como *unique*, lo que hace la carga idempotente (`truncate ...restart identity`). `dim_product` se construye con `LEFT JOIN` entre `product` (50 filas) y `central_product` (49 filas) — un producto carece de coste y su margen se gestiona con `coalesce(unit_cost, 0)`. `fact_returns` desnormaliza `customer_sk`, `product_sk` y `store_sk` desde `fact_sales` a través de `sale_item_id`, lo que evita *re-joins* en analítica de devoluciones. En el área *staging*

se recomputa `subtotal` en el 100% de las líneas porque 8 filas `raw` eran inconsistentes con `quantity × unit_price`; el test `test_revenue_matches_between_staging_and_dwh` certifica que la corrección no rompe la conservación de ingresos. El DDL completo está en `sql/02_dwh_schema/`.

3 ETL: del raw al DWH

El pipeline encadena tres saltos `public → staging → dwh`, orquestados por seis notebooks numerados que comparten utilidades reutilizables en `src/`. La tabla 2 resume cada paso.

Tabla 2: Notebooks que orquestan el pipeline y sus salidas.

#	Notebook	Salida
00	00_exploration.ipynb	perfilado de <code>public</code>
01	01_etl_staging.ipynb	tablas <code>staging.*</code> limpias
02	02_etl_dwh.ipynb	esquema <code>dwh</code> cargado
03	03_cltv_calculation.ipynb	<code>data/processed/cltv.parquet</code>
04	04_customer_metrics.ipynb	<code>customer_metrics.parquet</code>
05	05_pca_clustering.ipynb	<code>models/*.joblib + customer_segments.parquet</code>

Las transformaciones SQL viven en `sql/03_transformations/`, con prefijos numéricos que fijan el orden de ejecución (`10_load_dim_date.sql`, `20_load_dim_customer.sql`, ..., `70_load_fact_returns.sql`); son aplicadas por `src.etl.load.run_sql_directory`, que delega en un singleton SQLAlchemy expuesto por `src/db/engine.py`. La dimensión calendario se genera con `generate_series` entre la primera venta y la última devolución. `fact_sales` resuelve las cinco FKs vía `LEFT JOIN` con las dimensiones ya pobladas y calcula `cost` y `margin` con `coalesce(unit_cost, 0)` para manejar el producto sin coste; `fact_returns` reusa las FKs de `fact_sales` a través de `sale_item_id` y multiplica la cantidad devuelta por `unit_price` para obtener `returned_value`. La trazabilidad se canaliza por el logger común `src.utils.get_logger`.

Reproducibilidad

Tres comandos sobre el entorno `UAX` regeneran el proyecto desde cero. Tras configurar `.env` con las credenciales de PostgreSQL e instalar dependencias (`conda run -n UAX pip install -r requirements.txt`), una sola invocación de `jupyter nbconvert -to notebook -execute -inplace notebooks/0*.ipynb` ejecuta los seis notebooks en orden, repoblando `staging`, `dwh` y las salidas analíticas de `data/processed/` y `models/`.

4 CLTV y métricas de cliente

El enunciado fija el CLTV por cliente sobre una ventana temporal t como el producto multiplicativo

$$\text{CLTV}(c) = \text{Ingresos}_t(c) \times \text{Margen}_t(c) \times \text{Frecuencia}_t(c) \times R_t(c). \quad (1)$$

En la implementación, la ventana coincide con la cobertura efectiva del dataset, 2020-01 $\rightarrow$ 2026-01, esto es, $W = 72$ meses. La tabla 3 formaliza la traducción de cada componente a columnas de `cltv.parquet`. El módulo `src/analytics/cltv.py` aplica la fórmula como un producto elemento a elemento de las cuatro columnas; el test `test_cltv_formula_consistency` verifica la igualdad fila a fila sobre las primeras 50 filas.

Tabla 3: Operacionalización de la fórmula de CLTV sobre el DWH.

Componente	Definición
Ingresos_t	$\sum_l \text{subtotal}_l - \sum_r \text{returned_value}_r$ (<code>net_revenue</code>)
Margen_t	$(\text{Ingresos}_t - \sum_l \text{unit_cost}_l \cdot \text{quantity}_l) / \text{Ingresos}_t$ (<code>margin_rate</code>)
Frecuencia_t	$ \{\text{sale_id} : c\} $ (<code>frequency</code>)
R_t	<code>active_months</code> / W , $W = 72$ (<code>retention_rate</code>)

Resultados agregados

Para $n = 5\,750$ clientes, la tabla 4 resume los estadísticos por componente. La distancia entre mediana (0,72 €) y media (3 165 €) del CLTV refleja la asimetría extrema de la distribución (figura 3): el 87% de los clientes tiene frecuencia 1 y un pequeño grupo recurrente concentra todo el valor.

Tabla 4: Estadísticos por componente del CLTV ($n = 5\,750$, $W = 72$).

Magnitud	Mediana	Media	Máximo
Ingresos brutos (€)	149,9	1 683	24 922
Margen (proporción)	0,40	0,36	1,00
Frecuencia (tickets)	1	3,48	34
R_t	0,014	0,044	0,40
CLTV (€)	0,72	3 165	109 735

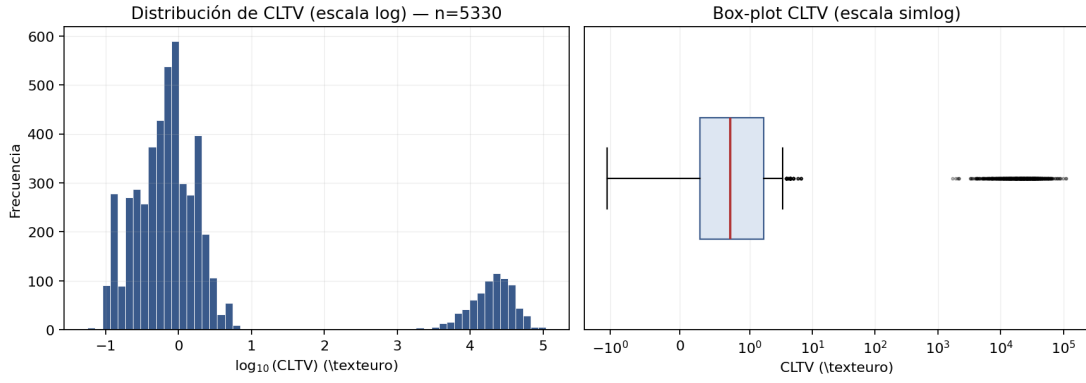


Figura 3: Distribución del CLTV en `cltv.parquet`: histograma en escala $\log_{10}$ (izquierda) y *box-plot* en escala simlog (derecha).

Métricas adicionales

El notebook 04 produce tres métricas complementarias por cliente en `customer_metrics.parquet`: *recencia* (mediana 421 d, $P75 \approx 1\,300$ d), *AOV* o ticket medio (mediana 129,99 €, media 210,04 €), y *tasa de devolución* (media 6,9%, $P95 = 1,0$). Los tests `test_metrics_aov_matches_revenue_div_frequency` y `test_recency_is_non_negative` formalizan sus dos invariantes algebraicas básicas: $AOV = \text{net_revenue} / \text{frequency}$ para frecuencia positiva, y $\text{recency} \geq 0$.

5 Segmentación: PCA + KMeans

La segmentación parte de un panel por cliente con ocho *features*: `net_revenue`, `margin_rate`, `frequency`, `retention_rate`, `aov`, `recency_days`, `return_rate` y `cltv`. El pipeline es la composición canónica $X \rightarrow Z = \text{StandardScaler}(X) \rightarrow Y = \text{PCA}_{n=2}(Z) \rightarrow \text{KMeans}_{k=3}(Y)$, encapsulada en `src/analytics/clustering.py`.

Fundamento matemático

Sea $S = \frac{1}{n-1} Z^\top Z$ la covarianza muestral. Las direcciones óptimas del PCA verifican

$$w_i = \arg \max_{\|w\|=1} w^\top S w, \quad (2)$$

cuya solución por las condiciones KKT da $S w_i = \lambda_i w_i$: los *vectores propios* de S ordenados por *valor propio* descendente. Las dos primeras componentes explican 57,1% y 23,2% de la varianza ($\Sigma \approx 80,3\%$), valores leídos del atributo `explained_variance_ratio_` de `models/pca.joblib`.

El KMeans minimiza la inercia intra-cluster

$$J(C_1, \dots, C_k) = \sum_{i=1}^k \sum_{y \in C_i} \|y - \mu_i\|_2^2, \quad \mu_i = \frac{1}{|C_i|} \sum_{y \in C_i} y, \quad (3)$$

mediante el algoritmo de Lloyd, entrenado sobre el espacio reducido $Y \in \mathbb{R}^{n \times 2}$. La curva del codo (figura 4) cae de 13 360 ($k = 2$) a 2 934 ($k = 3$) y desciende suavemente desde ahí; se fija $k^* = 3$.

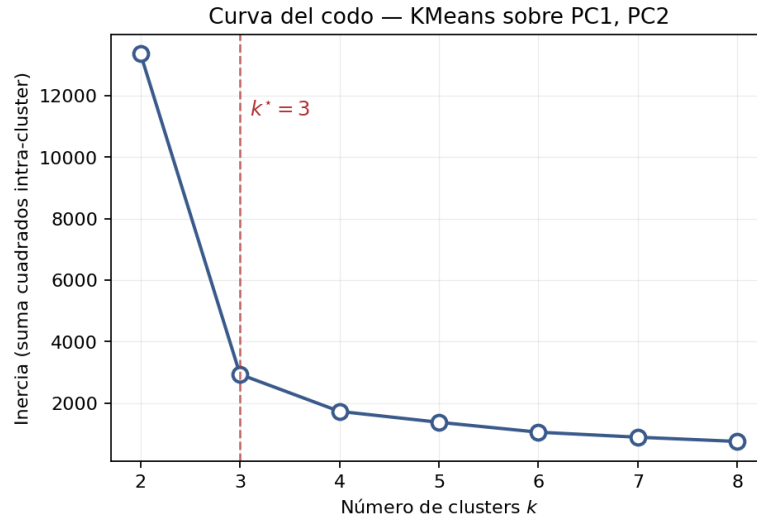


Figura 4: Inercia $J^*(k)$ frente a $k \in \{2, \dots, 8\}$. El codo en $k^* = 3$ justifica la elección final.

Resultado: perfiles de cluster

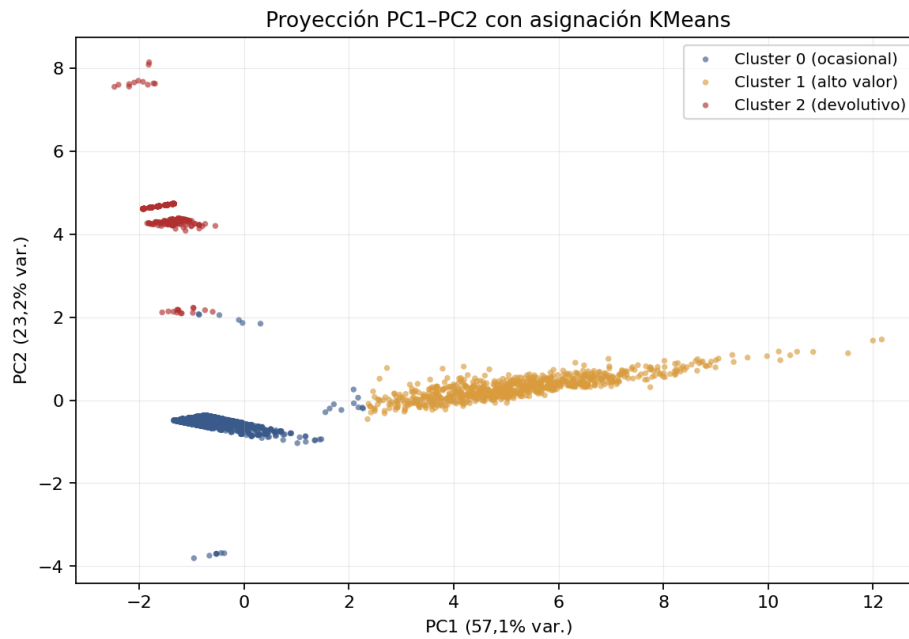


Figura 5: Proyección PC1-PC2 de los 5 750 clientes coloreada por cluster KMeans.

Tabla 5: Perfil mediano por cluster, leído de `models/customer_segments.parquet`.

Cluster	<i>n</i>	net_rev.	margin	freq.	ret.	aov	recency	ret_rate	CLTV
0 (<i>ocasional</i>)	4 576	129,90	0,400	1	0,014	129,90	854	0,000	0,72
1 (<i>alto valor</i>)	740	11 461,53	0,388	20	0,250	572,24	82	0,022	21 191,30
2 (<i>devolutivo</i>)	434	0,00	0,000	1	0,014	0,00	856	1,000	0,00

El *cluster 0* ($\approx 80\%$ de la base) recoge los clientes *one-shot*. El *cluster 1* ($\approx 13\%$) son recurrentes con mediana de 20 tickets, AOV de 572 € y retención del 25%, y concentran el grueso del CLTV agregado. El *cluster 2* ($\approx 7,5\%$) son clientes devolutivos con tasa de devolución mediana del 100% — el grupo separado por PC2, y ruido desde el punto de vista de valor.

6 Conclusiones

Hallazgos cuantitativos

Sobre 5 750 clientes, 20 000 tickets y 42 555 líneas de venta, el pipeline ETL puebla un DWH en estrella (6 dimensiones, 2 hechos) de forma reproducible: una sola invocación de `jupyter nbconvert --execute notebooks/0*` regenera los esquemas `staging` y `dwh`, los `parquet` de analítica y los modelos de `models/`. La distribución del CLTV es extremadamente asimétrica (mediana 0,72 € frente a media 3 165 €, máximo 109 735 €), con un coeficiente de Gini $\approx 0,91$: el 87% de los clientes son *one-shot* y el cluster “alto valor” (13% de la base) concentra el grueso del CLTV agregado. El segmentador PCA(2) + KMeans(3) recupera tres clusters interpretables (ocasionales, alto valor, devolutivos) con 80,3% de varianza explicada y un codo claro en $k^* = 3$.

Limitaciones

El dataset es *sintético* y algunas decisiones de diseño preservan los datos tal cual aunque generen artefactos: existen 10 clientes con `return_rate` > 1 (devoluciones que superan la venta original) y 1 producto sin `unit_cost` en `central_product`; ambos casos se conservan y se gestionan con `coalesce(..., 0)` en el cálculo de coste. La retención R_t es una proxy discreta (`active_months/72`) y el KMeans se entrena sobre el espacio reducido por PCA(2), que descarta un $\sim 20\%$ de la varianza original.

Líneas futuras y validación

Las extensiones naturales son: ampliar las *features* con señales territoriales (`district`, `area_type`) y promocionales (`discount_percent`); validar k con métricas de silueta y Davies–Bouldin; sustituir la fórmula multiplicativa por modelos probabilísticos tipo BG/NBD + Gamma–Gamma; y refactorizar el pipeline para ingestión incremental con *slowly changing dimensions*. La calidad queda anclada por nueve pruebas `pytest` en `tests/test_cltv.py` y `tests/test_dwh_integrity.py` (integridad referencial, unicidad de *business keys*, conservación de ingresos `staging` $\rightarrow$ `dwh`, y consistencia algebraica de la fórmula de CLTV).